

Sunbeam Institute of Information Technology
Pune and Karad

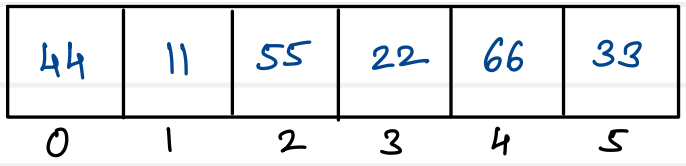
Algorithms and Data structures

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Selection sort

1. Select one position of the array
2. Find smallest element out of remaining elements
3. Swap selected position element and smallest element
4. Repeat above steps until array is sorted (N-1)

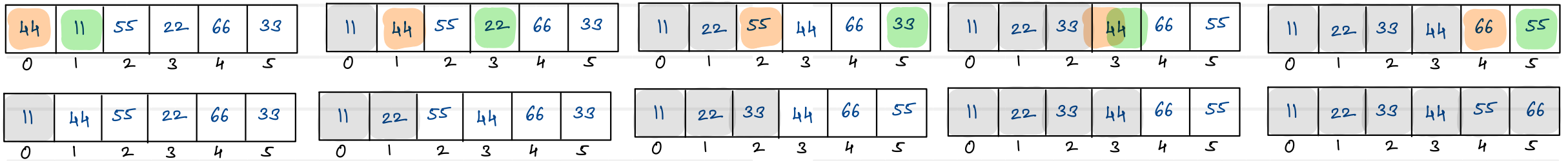


i	minIndex	j
2	~~2~~ ~~3~~ 5	3 4 5 6

i	minIndex	j
3	3	4 5 6

i	minIndex	j
4	~~4~~ 5	5 6

Pass 1 | Pass 2 | Pass 3 | Pass 4 | Pass 5

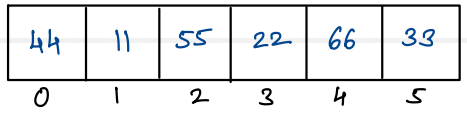


to select elements : $i = 0 \rightarrow N-2$ $(i < N-1)$

to find smallest out of remaining : $j = i+1 \rightarrow N-1$ $(j < N)$

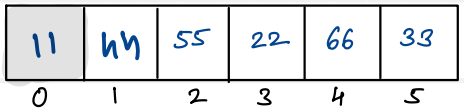
```
public static void selectionSort(int arr[], int N){
    //1. select positions one by one
    for( int i=0 ; i<N-1 ; i++){
        //2. find min element from remaining elements
        int minIndex = i;
        for(int j=i+1 ; j<N ; j++){
            if(arr[j] < arr[minIndex])
                minIndex = j;
        }
        //3. swap selected position element & smallest element
        int temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}
```

Selection sort



i	minIndex	j
0	0	1
	1	2
		3
		4
		5
		6 ✗

```
minIndex = i
for( j=i+1 ; j<N ; j++)
    if( arr[j] < arr[minIndex])
        minIndex = j;
```



i	minIndex	j
1	1	2
	3	3
		4
		5
		6 ✗

n - no. of elements

n-1 - no. of passes

passes	comps
1	n-1
2	n-2
⋮	⋮
n-1	1

Total comps = $1+2+3+\ldots+(n-1)$ (n terms)

$$= \frac{n(n+1)}{2}$$

$$= \frac{1}{2}(n^2+n)$$

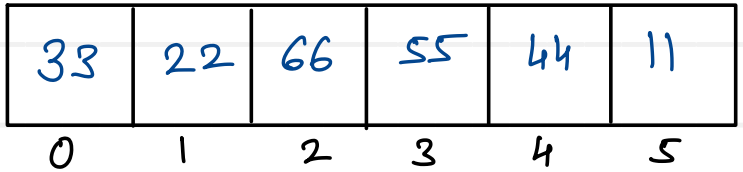
$$Time \propto \frac{1}{2}(n^2+n)$$

$$T(n) = O(n^2)$$ Best Avg worst

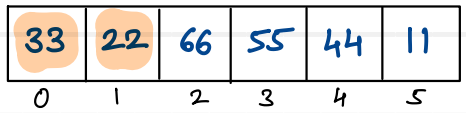
$$S(n) = O(1)$$

Bubble sort

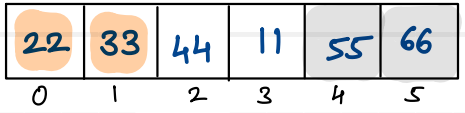
1. Compare all pairs of consecutive elements of the array one by one
2. If left element is greater than right element , then swap both
3. Repeat above steps until array is sorted (N-1)

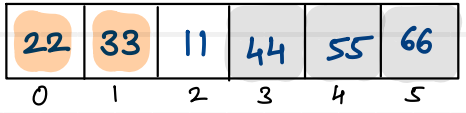


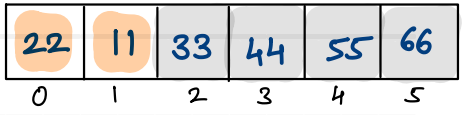
i ← j j+1 → r



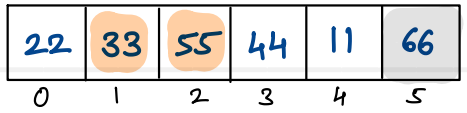


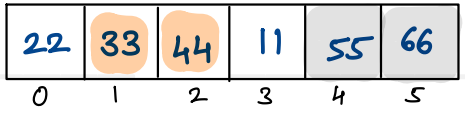


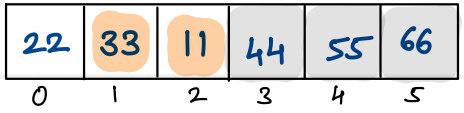


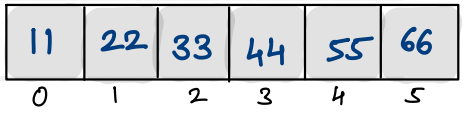


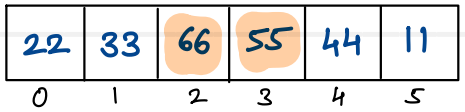


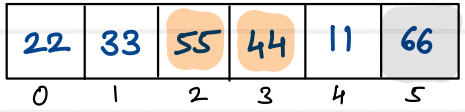


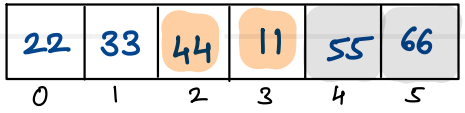


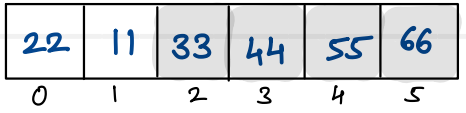




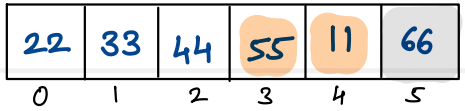


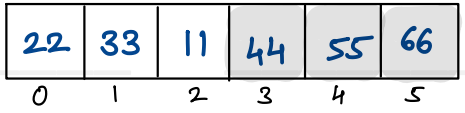




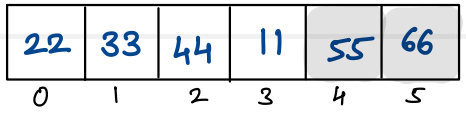


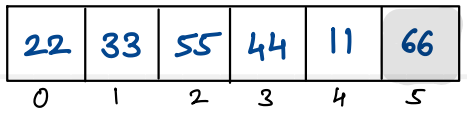












to repeat : i = 1 → N-1 (i < N)

i	j < N-1	j < N-i	
1	0 → 4	0 → 4	j < 5
2	0 → 4	0 → 3	j < 4
3	0 → 4	0 → 2	j < 3
4	0 → 4	0 → 1	j < 2
5	0 → 4	0 → 0	j < 1

Bubble sort

11	22	33	44	55	66
11	22	33	44	55	66
11	22	33	44	55	66
11	22	33	44	55	66
11	22	33	44	55	66

No. of comps = n-1

Time $\propto$ n-1

$T(n) = O(n)$ Best

n - no. of elements

n-1 - no. of passes

passes	comps
1	n-1
2	n-2
⋮	⋮
n-1	1

Total comps = $1+2+3+\ldots+(n-1)$ (n)

$= \frac{n(n+1)}{2}$

$= \frac{1}{2}(n^2+n)$

Time $\propto \frac{1}{2}(n^2+n)$

$T(n) = O(n^2)$ Avg worst

$S(n) = O(1)$

Insertion sort

1. Pick one element of the array (start from 2nd element)
2. Compare picked element with all its left neighbours one by one
3. If left neighbour is greater, move it one position ahead
4. Insert picked element at its appropriate position
5. Repeat above steps until array is sorted (N-1)

passes	comps
1	1
2	2
⋮	⋮
n-1	n-1

pass 1 : 11 22 33 44 55

pass 2 : 11 22 33 44 55

pass 3 : 11 22 33 44 55

pass 4 : 11 22 33 44 55

Total no. of comps = n-1

Time $\propto$ n-1

$T(n) = O(n)$ Best

Total comps = $1+2+3+\dots+(n-1)$ (n terms)

$= \frac{n(n+1)}{2}$

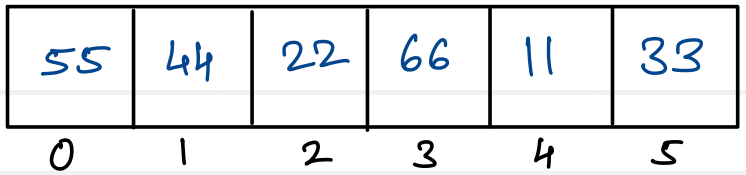
$= \frac{1}{2}(n^2+n)$

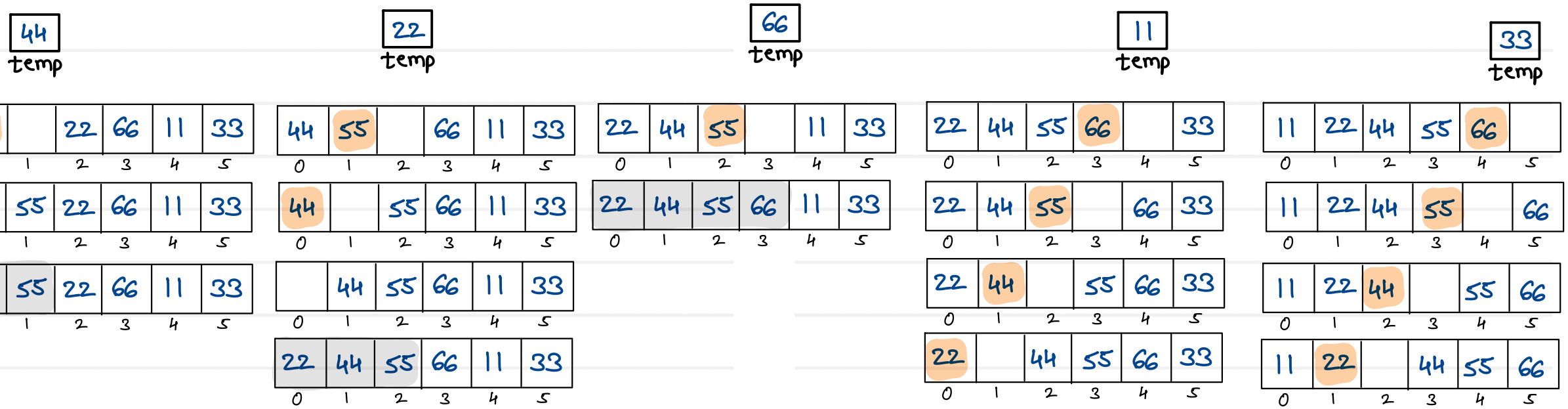
Time $\propto \frac{1}{2}(n^2+n)$

$T(n) = O(n^2)$ Avg worst

$S(n) = O(1)$

Insertion sort



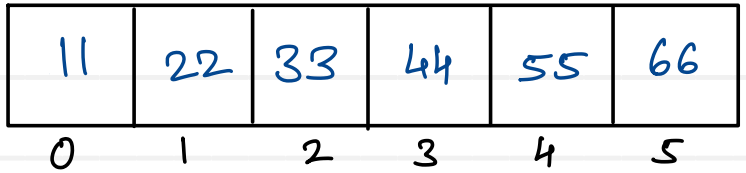


to pick elements : $i = 1 \rightarrow N-1$ $(i < N)$ $i++$

to compare with left neighbors : $j = i-1 \rightarrow 0$ $(j >= 0)$ $j--$

Insertion sort

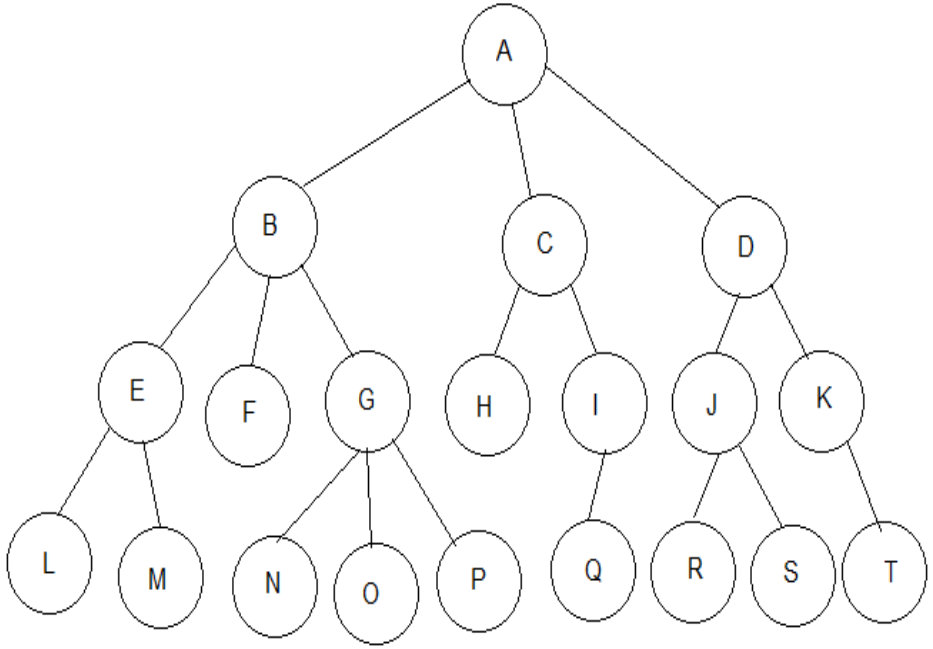
```
for( i=1 ; i<N ; i++) {
    temp = arr[i]
    j;
    for(j=i-1 ; j>=0; j--){
        if(arr[j] > temp)
            arr[j+1] = arr[j];
        else
            break;
    }
    arr[j+1] = temp;
}
```



i	i<6	temp	j
1	T	44	0,-1
2	T	22	1,0,-1
3	T	66	2
4	T	11	3,2,1,0,-1
5	T	33	4,3,2,1

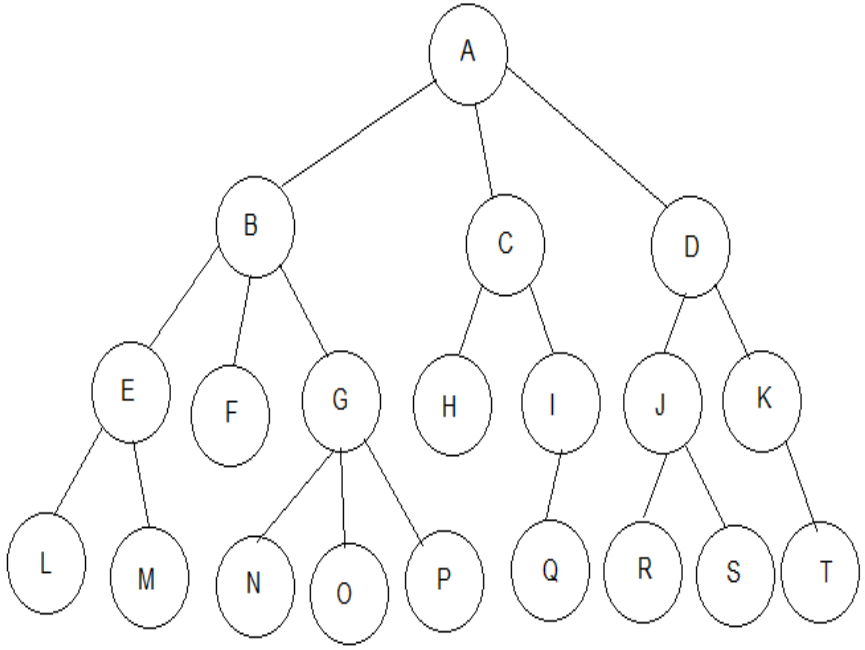
Tree - Terminologies

- **Tree** is a **non linear** data structure which is a finite set of nodes with one specially designated node is called as "**root**" and remaining nodes are partitioned into m disjoint subsets where each of subset is a tree..
- **Root** is a **starting point** of the tree.
- All nodes are connected in **Hierarchical manner (multiple levels)**.
- **Parent node:-** having other child nodes connected
- **Child node:-** immediate descendant of a node
- **Leaf node:-**
  - Terminal node of the tree.
  - Leaf node does not have child nodes.
- **Ancestors:-** all nodes in the path from root to that node.
- **Descendants:-** all nodes accessible from the given node
- **Siblings:-** child nodes of the same parent



Tree - Terminologies

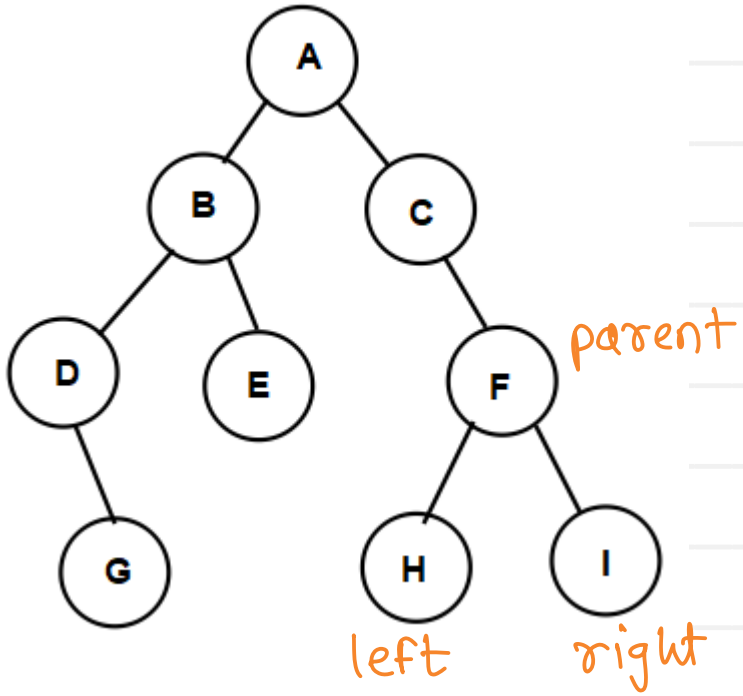
- **Degree of a node :-** number of child nodes for any given node.
- **Degree of a tree :-** Maximum degree of any node in tree.
- **Level of a node :-** indicates position of the node in tree hierarchy
  - Level of child = Level of parent + 1
  - Level of root = 0
- **Height of node :-** number of links from node to longest leaf.
- **Depth of node :-** number of links from root to that node
- **Height of a tree :-** Maximum height of a node (height of root)
- **Depth of a tree :-** Maximum depth of a node



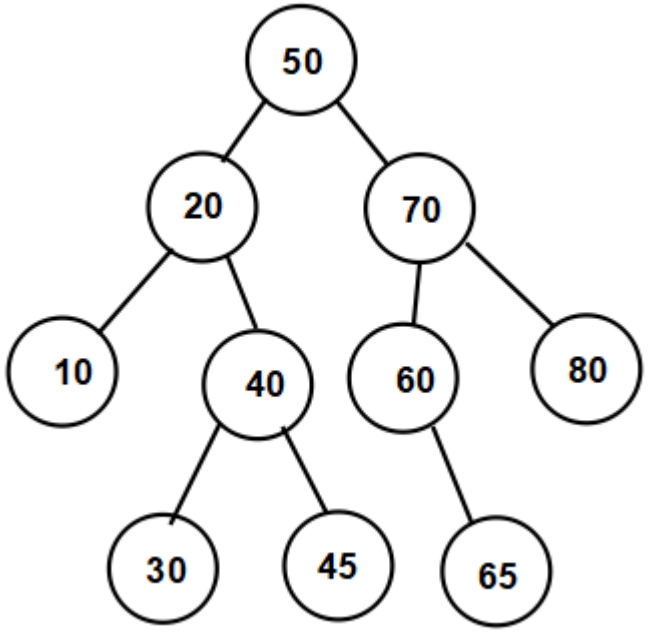
- Tree with zero nodes (ie empty tree) is called as "**Null tree**". Height of Null tree is -1.
  - Tree can grow up to any level and any node can have any number of Childs.
  - That's why operations on tree becomes un efficient.
  - Restrictions can be applied on it to achieve efficiency and hence there are different types of trees.

Tree - Terminologies

- **Binary Tree**
  - Tree in which each node has maximum two child nodes
  - Binary tree has degree 2. Hence it is also called as 2- tree



- **Binary Search Tree**
  - Binary tree in which left child node is always smaller and right child node is always greater or equal to the parent node.
  - Searching is faster
  - Time complexity : O(h) h – height of tree



Binary Search Tree - Implementation

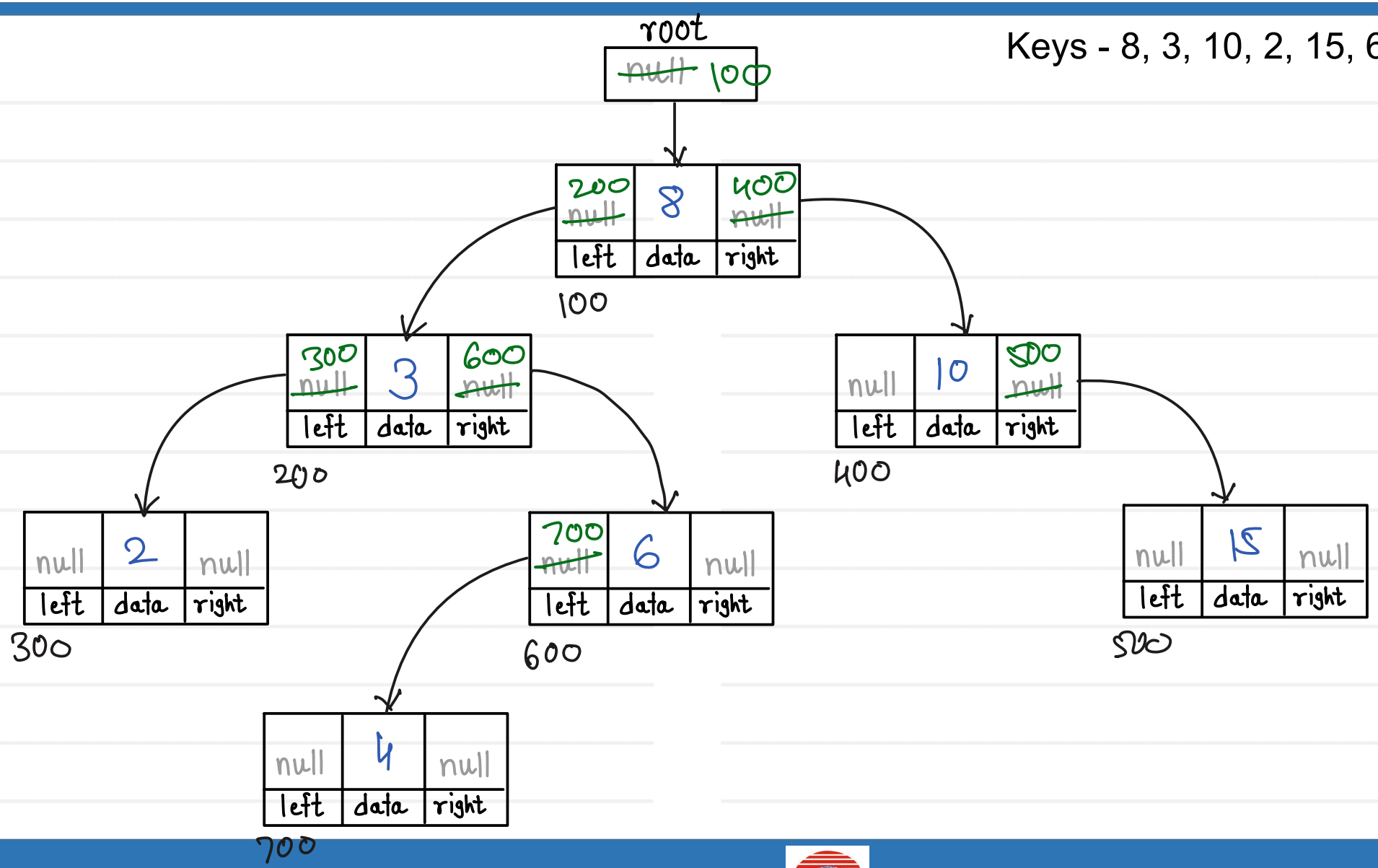
Node :
- data - actual data
- left - reference of left child
- right - reference of right child

```
class Node {
    int data;
    Node left;
    Node right;
}
```

```
class BST {
    static class Node {
        int data;
        Node left;
        Node right;
    }
    Node root;
    public BST() {...}
    public addNode(value) {...}
    public deleteNode(value) {...}
    public searchNode(key) {...}
    public traverse() {...}
    public deleteAll() {...}
}
```

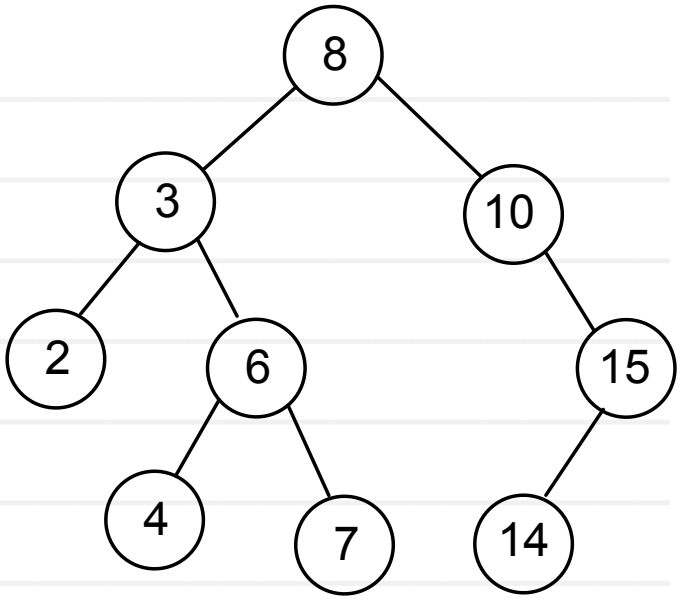
Binary Search Tree - Add Node

Keys - 8, 3, 10, 2, 15, 6, 14, 4, 7



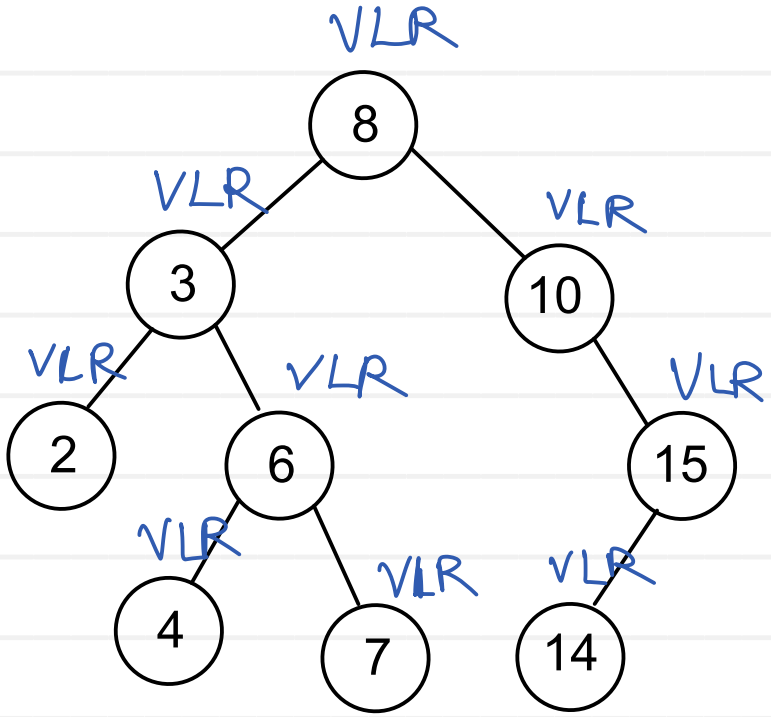
Binary Search Tree – Add Node

```
//1. create node for given value
//2. if BSTree is empty
      // add newnode into root itself
//3. if BSTree is not empty
      //3.1 create trav reference and start at root node
      //3.2 if value is less than current node data (trav.data)
            //3.2.1 if left of current node is empty
                  // add newnode into left of current node
            //3.2.2 if left of current node is not empty
                  // go into left of current node
      //3.3 if value is greater or equal than current node data (trav.data)
            //3.3.1 if right of current node is empty
                  // add newnode into right of current node
            //3.3.2 if right of current node is not empty
                  // go into right of current node
      //3.4 repeat step 3.2 and 3.3 till node is not getting added into BSTree
```



Tree Traversal Techniques

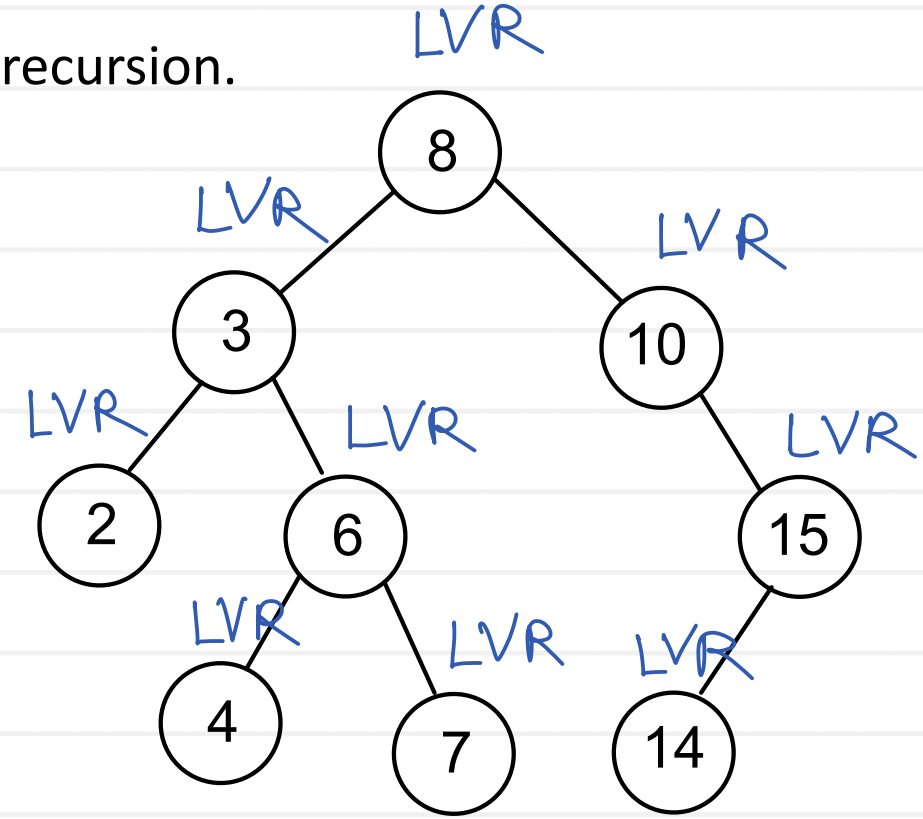
- **Pre-Order:-** V L R
- **In-order:-** L V R
- **Post-Order:-** L R V
- The traversal algorithms can be implemented easily using recursion.
- Non-recursive algorithms for implementing traversal needs stack to store node pointers.



- **Pre-Order** :- 8, 3, 2, 6, 4, 7, 10, 15, 14

Tree Traversal Techniques

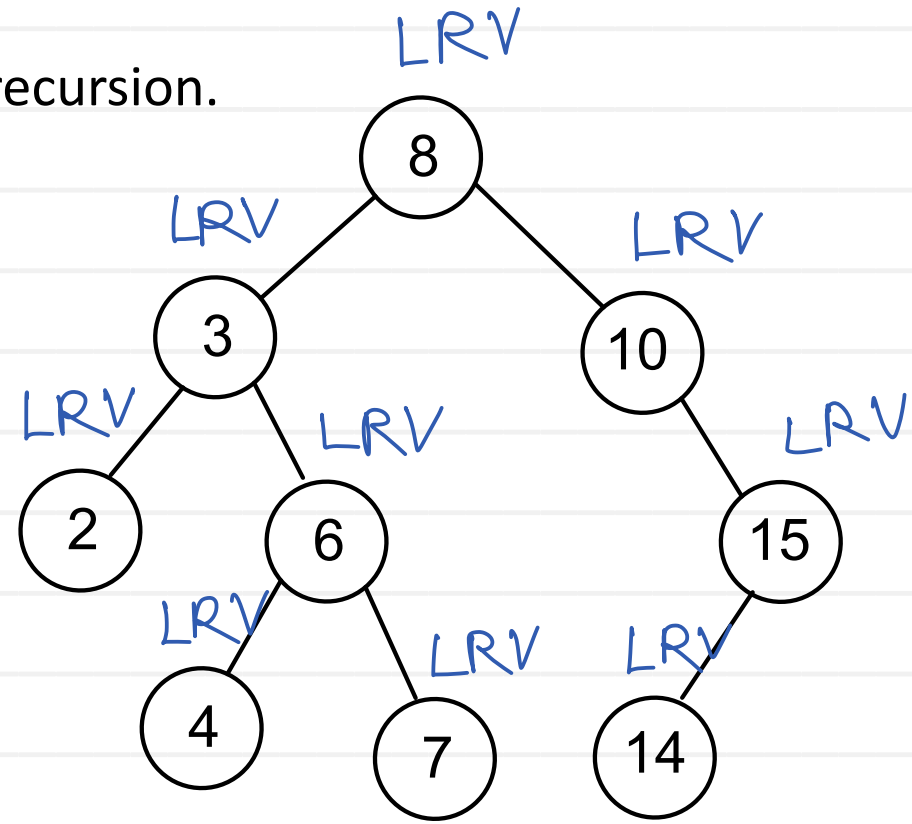
- **Pre-Order:-** V L R
- **In-order:-** L V R
- **Post-Order:-** L R V
- The traversal algorithms can be implemented easily using recursion.
- Non-recursive algorithms for implementing traversal needs stack to store node pointers.



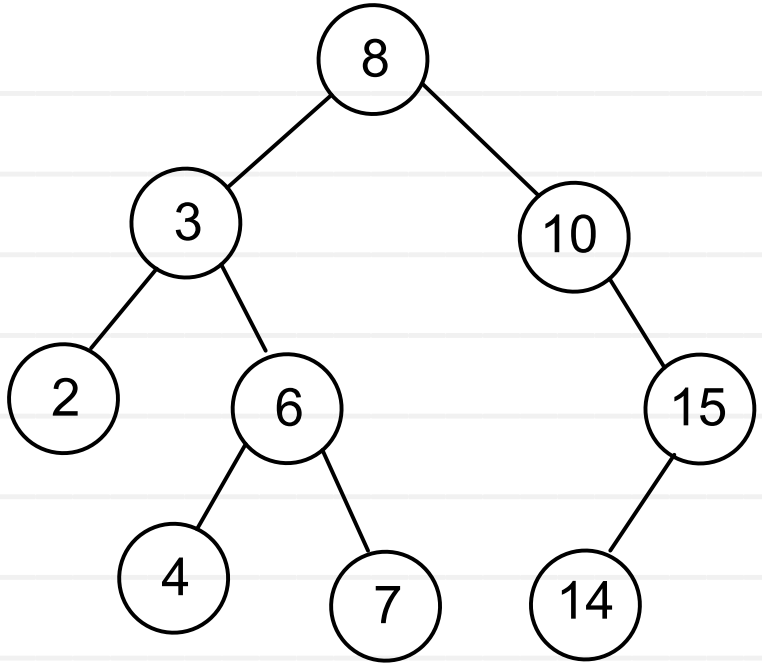
- **In-Order** :- 2, 3, 4, 6, 7, 8, 10, 14, 15 (sorted)

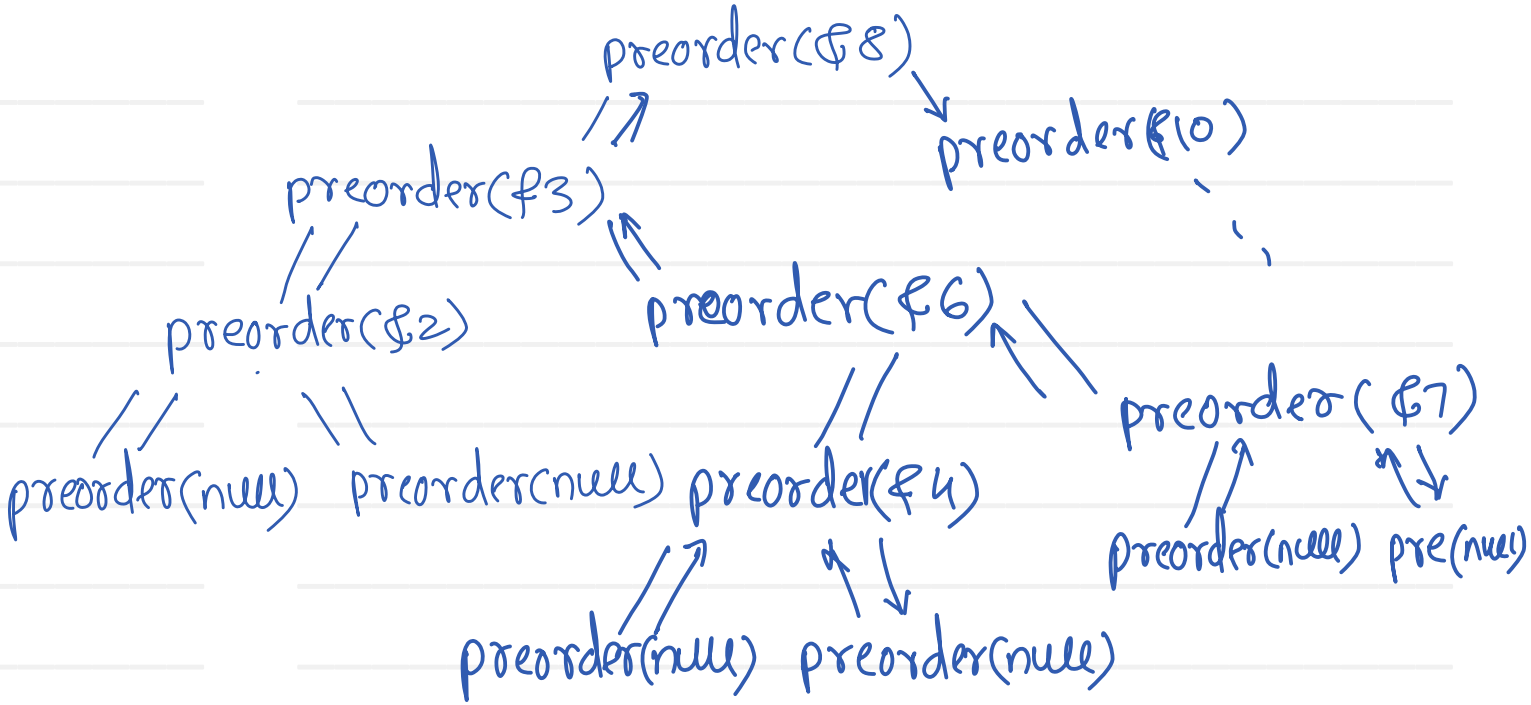
Tree Traversal Techniques

- **Pre-Order:-** V L R
- **In-order:-** L V R
- **Post-Order:-** L R V
- The traversal algorithms can be implemented easily using recursion.
- Non-recursive algorithms for implementing traversal needs stack to store node pointers.



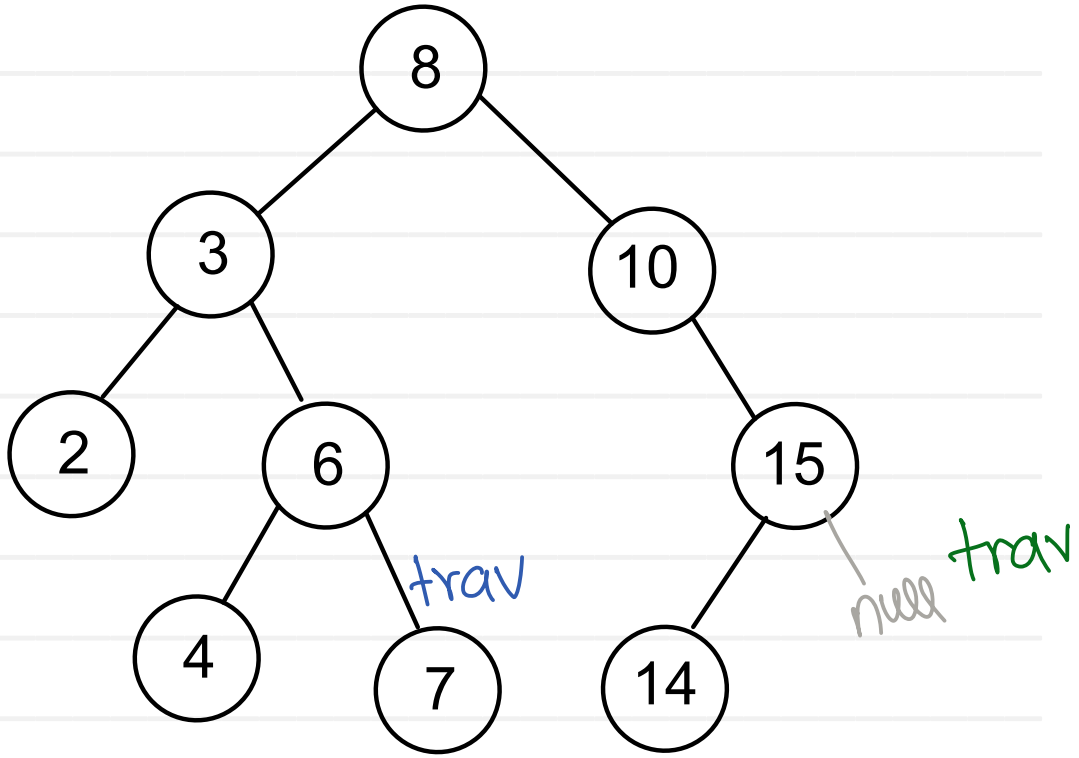
- **Post-Order** :- 2, 4, 7, 6, 3, 14, 15, 10, 8






```
pre order(trav) {
    if(trav == null)
        return;
sysout(trav.data);
pre order(trav.left);
preorder(trav.right);
}
```

Binary Search Tree - Binary Search



1. Start from root
2. If key is equal to current node data return current node
3. If key is less than current node data search key into left sub tree of current node
4. If key is greater than current node data search key into right sub tree of current node
5. Repeat step 2 to 4 till leaf node

key = 7

trav

8 <

3 >

6 >

7 =

key = 16

trav

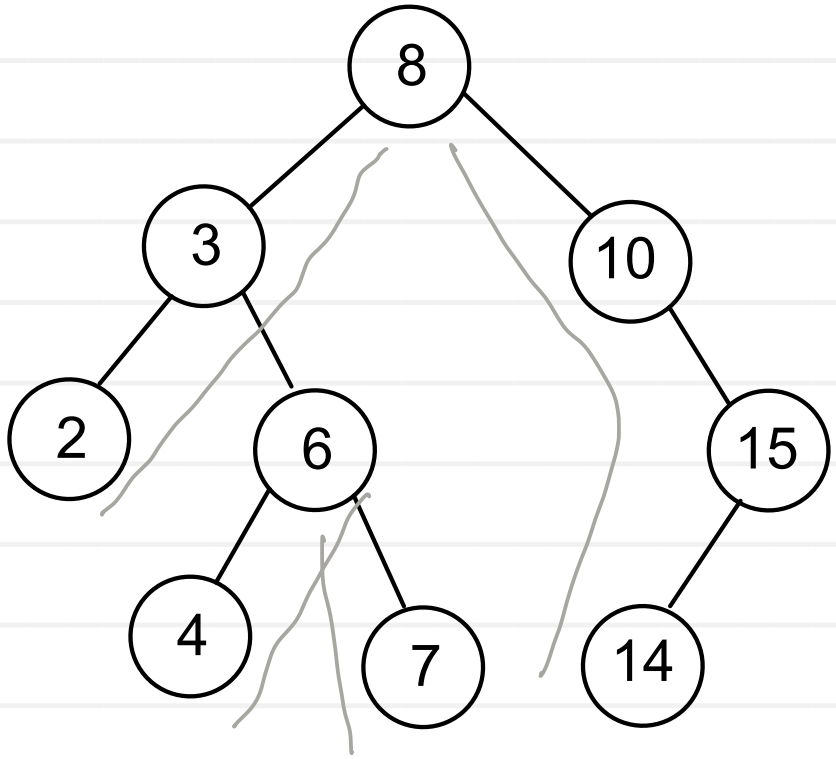
8 >

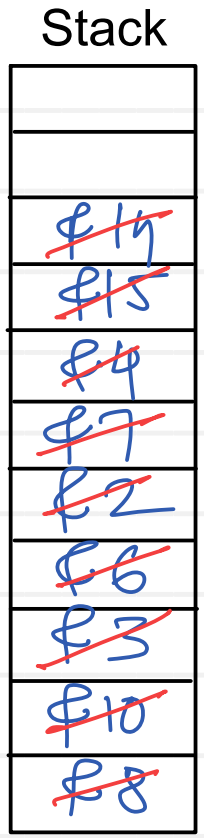
10 >

15 >

null

Binary Search Tree - DFS Traversal (Depth First Search)

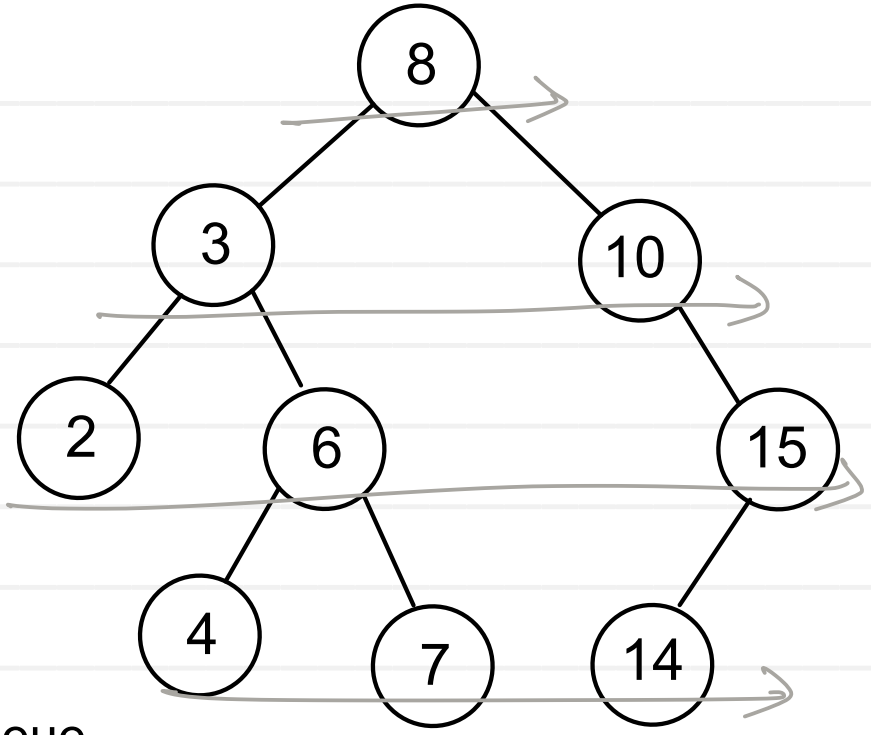




1. Push root node on stack
2. Pop one node from stack
3. Visit (print) popped node
4. If right exists, push it on stack
5. If left exists, push it on stack
6. While stack is not empty, repeat step 2 to 5

Traversal : 8, 3, 2, 6, 4, 7, 10, 15, 14

Binary Search Tree - BFS Traversal (Bredth First Search)



1. Push root node on queue
2. Pop one node from queue
3. Visit (print) popped node
4. If left exists, push it on queue
5. If right exists, push it on queue
6. While queue is not empty, repeat step 2 to 5

level order traversal

Queue

8	3	10	2	6	15	4	7	14

Traversal : 8, 3, 10, 2, 6, 15, 4, 7, 14